비개발자를 위한 Claude Code 하네스 가이드

실밸개발자 채널 내용을 비개발자용으로 정리했습니다.

0. 이 가이드의 목적

이 가이드는 코딩을 잘 모르는 사람이 Claude Code를 이용해 앱, 웹서비스, 자동화 도구, 문서 시스템 등을 만들 때 필요한 **운영 체계**입니다.

중요한 관점은 이것입니다.

> 비개발자는 코드를 직접 고치는 사람이 아니라,
> **AI가 잘 일하도록 문서·규칙·절차를 설계하는 사람**이 되어야 한다.

Claude Code를 그냥 쓰면 "이 기능 추가해줘", "버그 고쳐줘", "다시 만들어줘" 식으로 흐르기 쉽습니다.
그렇게 하면 기능은 늘어나지만 구조는 무너지고, 나중에는 무엇을 어떻게 고쳐야 하는지 알 수 없게 됩니다.

그래서 필요한 것이 **하네스**입니다.

1. 하네스란 무엇인가?

하네스는 Claude Code가 마음대로 판단하지 못하도록 잡아주는 **작업 제어 시스템**입니다.

쉽게 말하면:

구분	설명
일반 프롬프트	요청 → 결과 → 끝
하네스 방식	문서 → 규칙 → 작업 분류 → 실행 → 검토 → 기록 → 반복

하네스의 목적은 Claude를 더 똑똑하게 만드는 것이 아닙니다.

> Claude를 **통제 가능하게 만드는 것**입니다.

2. 최종 추천 폴더 구조

아래 구조를 기본으로 쓰면 됩니다.

```
my-claude-project/
  CLAUDE.md

  docs/
    PRD.md
    ARCHITECTURE.md
    ADR.md
    UI_GUIDE.md
    TEST_PLAN.md

  HARNESS/
    router.md
    context.md
    loop.md
    roles.md

  TODO.md
  LOG.md

  source/
  output/
```

각 파일의 역할은 다음과 같습니다.

파일/폴더	역할
CLAUDE.md	Claude의 절대 행동 규칙
docs/PRD.md	무엇을 만들지 정의
docs/ARCHITECTURE.md	구조와 기술 방향 정의
docs/ADR.md	주요 의사결정 기록
docs/UI_GUIDE.md	화면·디자인 기준
docs/TEST_PLAN.md	검증 기준

HARNESS/router.md	요청 분류
HARNESS/context.md	사용할 정보 제한
HARNESS/loop.md	계획→실행→검토→수정 반복
HARNESS/roles.md	작성자·검토자·수정자 역할 분리
TODO.md	작업 큐
LOG.md	작업 이력
source/	원본 자료
output/	결과물

3. 전체 작동 방식

Claude Code 작업은 항상 아래 순서로 진행합니다.

1. docs/에서 방향 확인
2. CLAUDE.md에서 행동 규칙 확인
3. TODO.md에서 다음 작업 선택
4. router.md로 작업 유형 분류
5. context.md로 사용할 정보 선택
6. loop.md로 계획 → 실행 → 검토 → 수정
7. roles.md로 품질 검토
8. TODO.md 업데이트
9. LOG.md 업데이트
10. output/에 결과 저장

한 줄로 요약하면:

> **docs가 방향을 정하고, CLAUDE.md가 행동을 제한하고, HARNESS가 실행을 통제하고, TODO/LOG가 연속성을 만든다.**

4. docs 폴더: 제품의 방향을 고정하는 곳

비개발자에게 가장 중요한 것은 코드가 아니라 **문서**입니다.

Claude Code는 문서를 보고 코드를 만듭니다.
따라서 문서가 부실하면 결과도 흔들립니다.

4.1 docs/PRD.md

PRD는 Product Requirements Document입니다.
쉽게 말하면 **제품 기획서**입니다.

예시:

```
# PRD

## 제품 개요

유튜브 URL을 입력하면 댓글을 수집하고 감성 분석 결과를 보여주는 웹 서비스

## 핵심 사용자

- 유튜브 크리에이터
- 마케팅 담당자
- 콘텐츠 분석 담당자

## 핵심 기능

- 유튜브 URL 입력
- 댓글 최대 100개 수집
- 긍정/부정/중립 감성 분석
- 결과 요약 표시
- 간단한 그래프 제공

## MVP 범위

이번 버전에서 만들 기능:

- URL 입력 화면
- 댓글 수집
```

```
- 감성 분석
- 결과 표시

## MVP 제외 범위

이번 버전에서 만들지 않을 기능:

- 로그인
- 결제
- 댓글 무제한 수집
- 실시간 업데이트
- 팀 협업 기능
- 모바일 앱
```

여기서 가장 중요한 부분은 **MVP 제외 범위**입니다.

비개발자는 보통 "이것도 추가해줘"를 반복하다가 프로젝트를 망칩니다.
그래서 처음부터 **안 만들 것**을 분명히 적어야 합니다.

4.2 docs/ARCHITECTURE.md

이 문서는 구조를 정의합니다.

예시:

```
# Architecture

## 기본 구조

- Frontend: Next.js
- Backend: Next.js API Routes
- AI 분석: Claude API
- Database: 초기 MVP에서는 사용하지 않음

## 데이터 흐름

User
→ URL 입력
→ API Route 호출
→ YouTube 댓글 수집
```

```
→ Claude API로 감성 분석
→ 결과 화면 표시

## 폴더 구조

app/
  page.tsx
  api/
    analyze/
      route.ts
components/
  ResultCard.tsx
lib/
  youtube.ts
  sentiment.ts
```

비개발자는 이 내용을 완벽히 이해할 필요는 없습니다.
하지만 Claude Code가 구조를 마음대로 바꾸지 못하게 하려면 필요합니다.

4.3 docs/ADR.md

ADR은 Architecture Decision Record입니다.
쉽게 말하면 **왜 그렇게 결정했는지 기록하는 문서**입니다.

예시:

```
# ADR

## ADR-001: Next.js 사용

### 결정
MVP는 Next.js로 만든다.

### 이유
- 프론트엔드와 API를 한 프로젝트에서 관리 가능
- Claude Code가 다루기 쉬움
- 배포가 쉬움

### 대안
- React + Express
- Python FastAPI

### 대안을 선택하지 않은 이유
```

```
초기 MVP에는 구조가 복잡해진다.
```

ADR이 없으면 Claude가 나중에 구조를 마음대로 바꿀 가능성이 커집니다.

4.4 docs/UI_GUIDE.md

UI가 마음에 안 드는 경우 대부분 원인은 Claude가 디자인 기준을 몰랐기 때문입니다.

예시:

```
# UI Guide

## 디자인 방향

- 단순하고 깔끔한 SaaS 스타일
- 흰 배경
- 카드 기반 레이아웃
- 너무 많은 색상 사용 금지

## 화면 구성

1. 상단 제목
2. URL 입력창
3. 분석 버튼
4. 결과 요약 카드
5. 감성 분석 그래프

## 금지 사항

- 복잡한 애니메이션 금지
- 과도한 색상 사용 금지
- 불필요한 사이드바 금지
```

4.5 docs/TEST_PLAN.md

비개발자도 테스트 기준은 적어야 합니다.

예시:

```
# Test Plan
```

```
## 기본 검증

- URL 입력창이 보이는가?
- 분석 버튼을 누르면 API가 호출되는가?
- 잘못된 URL 입력 시 에러 메시지가 나오는가?
- 분석 결과가 화면에 표시되는가?

## 금지되는 결과

- 빈 화면
- 무한 로딩
- API 키 노출
- 콘솔 에러
```

5. CLAUDE.md: Claude의 행동 규칙

CLAUDE.md는 Claude Code의 헌법입니다.

예시:

```
# CLAUDE.md

## 기본 원칙

CRITICAL: docs/PRD.md의 MVP 범위를 벗어난 기능은 만들지 않는다.
CRITICAL: 구조 변경 전에는 먼저 제안하고 승인받는다.
CRITICAL: API 키는 절대 코드에 하드코딩하지 않는다.
CRITICAL: 환경변수는 .env.local을 사용한다.
CRITICAL: 작업 후 변경 파일 목록을 보고한다.
CRITICAL: 작업 후 검증 결과를 보고한다.
CRITICAL: TODO.md의 다음 미완료 항목 하나만 처리한다.
CRITICAL: 작업 완료 후 LOG.md를 업데이트한다.

## 작업 방식

1. 작업 전 관련 문서를 읽는다.
2. 사용할 문서와 제외할 문서를 선언한다.
```

```
3. 계획을 먼저 제시한다.
4. 작은 단위로 수정한다.
5. 수정 후 자체 검토한다.
6. TODO.md와 LOG.md를 업데이트한다.

## 금지 사항

- 요청하지 않은 기능 추가 금지
- 전체 구조 임의 변경 금지
- 불필요한 리팩토링 금지
- 테스트 없이 완료 선언 금지
- 모르는 내용을 아는 척하지 말 것
```

비개발자는 이 파일을 자주 고쳐야 합니다.
문제가 생기면 코드를 먼저 고치는 것이 아니라, **CLAUDE.md나 docs를 고치는 것**이 원칙입니다.

6. HARNESS 폴더: Claude의 실행을 통제하는 장치

HARNESS/는 Claude가 작업을 어떻게 수행할지 정하는 실행 장치입니다.

6.1 HARNESS/router.md

요청을 분류합니다.

```
# Router

작업 시작 전 요청을 다음 중 하나로 분류한다.

## A. 단순 답변
코드 수정이나 파일 변경 없이 설명만 하면 되는 경우

## B. 질문 필요
요청이 모호하거나 필수 정보가 부족한 경우

## C. 작업 실행
파일 생성, 수정, 코드 작성, 문서 작성이 필요한 경우

## 규칙
```

```
- C로 분류되면 context.md를 실행한다.
- B로 분류되면 바로 작업하지 말고 질문한다.
- A로 분류되면 간단히 답변한다.
```

6.2 HARNESS/context.md

필요한 정보만 보게 합니다.

```
# Context

작업 시작 전 다음을 선언한다.

## 사용할 정보

- 어떤 docs 파일을 사용할지
- 어떤 source 파일을 사용할지
- 어떤 기존 코드를 참고할지

## 제외할 정보

- 이번 작업과 무관한 파일
- MVP 범위 밖 기능
- 과거에 폐기된 결정

## 규칙

- 모든 파일을 무작정 읽지 않는다.
- 필요한 정보만 사용한다.
- PRD와 충돌하면 PRD를 우선한다.
- ADR과 충돌하면 ADR을 확인한다.
```

이것이 중요한 이유는 Claude가 너무 많은 정보를 보면 오히려 중요한 제약을 놓칠 수 있기 때문입니다.

6.3 HARNESS/loop.md

작업 반복 루프입니다.

```
# Loop

모든 작업은 아래 순서로 수행한다.

## 1. Plan

- 목표 확인
- 수정할 파일 예상
- 위험 요소 확인

## 2. Draft

- 초안 또는 1차 구현 생성

## 3. Review

다음 기준으로 검토한다.

- PRD 범위 안인가?
- CLAUDE.md 규칙을 지켰는가?
- 구조가 단순한가?
- 불필요한 기능이 추가되지 않았는가?
- 테스트 또는 검증이 가능한가?

## 4. Revise

검토 결과를 반영해 수정한다.

## 5. Report

작업 결과를 보고한다.

- 변경 파일
- 변경 내용
- 검증 결과
- 남은 문제
```

6.4 HARNESS/roles.md

Claude가 여러 역할을 번갈아 수행하게 합니다.

```
# Roles

작업 품질을 높이기 위해 다음 역할을 순서대로 수행한다.

## Writer

- 요청에 맞는 초안 또는 구현을 만든다.

## Reviewer

다음 관점으로 문제를 찾는다.

- 요구사항 누락
- 과도한 기능 추가
- 구조 문제
- 보안 문제
- 비개발자가 이해하기 어려운 부분

## Reviser

Reviewer의 지적을 반영해 수정한다.

## 규칙

최종 답변에는 다음을 포함한다.

1. 수행한 작업
2. 발견한 문제
3. 수정한 내용
4. 남은 리스크
```

7. TODO.md: 작업을 작게 나누는 실행 큐

`TODO.md`는 Claude Code가 다음에 무엇을 해야 할지 보는 작업 목록입니다.

예시:

```
# TODO
```

```
## 현재 목표

유튜브 댓글 감성 분석 MVP를 만든다.

## 작업 목록

- [ ] PRD.md 작성
- [ ] ARCHITECTURE.md 작성
- [ ] ADR.md 작성
- [ ] UI_GUIDE.md 작성
- [ ] TEST_PLAN.md 작성
- [ ] 초기 Next.js 프로젝트 구조 확인
- [ ] URL 입력 화면 구현
- [ ] API route 설계
- [ ] 댓글 수집 함수 작성
- [ ] 감성 분석 함수 작성
- [ ] 결과 화면 구현
- [ ] 기본 테스트
- [ ] LOG.md 업데이트
```

핵심 규칙은 이것입니다.

> Claude에게 한 번에 여러 작업을 시키지 말고,
> **다음 미완료 항목 하나만 처리하게 한다.**

8. LOG.md: 작업 기억 장치

`LOG.md`는 Claude의 작업 이력입니다.

예시:

```
# LOG

## 2026-05-01 10:30

### 처리한 TODO
- PRD.md 작성

### 사용한 자료
```

```
- 사용자 아이디어
- docs 템플릿

### 만든 결과물
- docs/PRD.md

### 검토 결과
- MVP 범위와 제외 범위가 정의됨
- 로그인, 결제, 실시간 업데이트 제외

### 남은 작업
- ARCHITECTURE.md 작성
- ADR.md 작성

### 다음 추천 작업
- ARCHITECTURE.md 작성
```

LOG가 있어야 세션이 바뀌어도 작업 맥락을 복구할 수 있습니다.

9. 실제 Claude Code 시작 명령

처음 시작할 때는 이렇게 입력합니다.

```
CLAUDE.md, docs/, HARNESS/, TODO.md, LOG.md를 읽어줘.

작업 방식은 다음을 따라줘.

1. router.md로 요청을 분류
2. context.md로 사용할 정보와 제외할 정보 선언
3. loop.md에 따라 계획 → 실행 → 검토 → 수정 진행
4. roles.md에 따라 작성자 → 검토자 → 수정자 역할 수행
5. TODO.md의 다음 미완료 항목 하나만 처리
6. 작업 완료 후 TODO.md와 LOG.md 업데이트

한 번에 모든 일을 하지 말고, 다음 미완료 TODO 하나만 처리해줘.
```

10. 매번 반복해서 쓰는 명령

작업을 계속 이어갈 때는 이것만 입력하면 됩니다.

```
TODO.md를 읽고 다음 미완료 항목 하나를 처리해줘.

작업 전에는 관련 docs와 CLAUDE.md, HARNESS 규칙을 확인하고,
작업 후에는 TODO.md와 LOG.md를 업데이트해줘.
```

11. 작업 점검 명령

Claude가 이상한 방향으로 가는 것 같으면 이렇게 물어봅니다.

지금 어느 단계인지 알려줘.
router / context / loop / roles 중 어디에 있는지 말해줘.

또는:

이번 작업에서 사용한 문서와 제외한 문서를 context.md 기준으로 다시 선언해줘.

또는:

loop.md 기준으로 다시 검토하고 수정해줘.

또는:

roles.md 기준으로 작성자, 검토자, 수정자 역할을 분리해서 다시 진행해줘.

12. 비개발자가 반드시 지켜야 할 운영 원칙

원칙 1. 코드를 먼저 고치지 말고 문서를 고친다

나쁜 방식:

이 코드 이상한데 고쳐줘.

좋은 방식:

PRD.md의 MVP 범위와 UI_GUIDE.md 기준에 맞지 않는 부분을 찾아서,
먼저 문서 기준으로 문제를 설명한 뒤 수정해줘.

원칙 2. 기능 추가보다 범위 제한이 중요하다

나쁜 방식:

로그인도 넣고, 결제도 넣고, 관리자 페이지도 넣어줘.

좋은 방식:

이번 MVP에서는 로그인, 결제, 관리자 페이지를 제외한다.
PRD.md의 MVP 범위 안에서만 구현해줘.

원칙 3. 한 번에 하나만 시킨다

나쁜 방식:

전체 앱을 완성해줘.

좋은 방식:

TODO.md의 다음 미완료 항목 하나만 처리해줘.

원칙 4. Claude가 한 일을 기록하게 한다

작업 후 반드시 LOG.md를 업데이트하게 해야 합니다.

이번 작업에서 변경한 파일, 변경 이유, 검증 결과, 남은 문제를 LOG.md에 기록해줘.

원칙 5. 구조 변경은 반드시 승인 후 진행하게 한다

CLAUDE.md에 이 규칙을 넣어야 합니다.

CRITICAL: 폴더 구조, 데이터 구조, API 구조를 바꾸기 전에는 먼저 변경 제안을 하고 승인받는다.

13. 문제 상황별 대응법

문제 1. Claude가 기능을 너무 많이 추가한다

원인:

- PRD의 MVP 제외 범위가 약함
- CLAUDE.md 규칙이 약함

대응:

PRD.md의 MVP 제외 범위를 강화해줘.
이번 버전에서 만들지 않을 기능을 명확히 추가해줘.
그 후 현재 구현이 MVP 범위를 벗어났는지 검토해줘.

문제 2. UI가 마음에 안 든다

원인:

- UI_GUIDE.md가 없음
- 디자인 기준이 모호함

대응:

UI_GUIDE.md를 먼저 작성해줘.
단순한 SaaS 스타일, 카드 기반 레이아웃, 과도한 색상 금지를 기준으로 정의해줘.
그 후 현재 UI를 UI_GUIDE.md 기준으로 수정해줘.

문제 3. 구조가 계속 바뀐다

원인:

- ARCHITECTURE.md와 ADR.md가 약함

대응:

ARCHITECTURE.md와 ADR.md 기준으로 현재 구조 변경이 필요한지 검토해줘.

구조 변경이 필요하다면 먼저 제안만 하고, 승인 전에는 수정하지 마.

문제 4. Claude가 이전 작업을 잊는다

원인:

- LOG.md가 없거나 업데이트되지 않음

대응:

LOG.md를 읽고 현재까지의 작업 이력을 요약해줘.
그 후 TODO.md의 다음 미완료 항목 하나만 처리해줘.

문제 5. 결과가 점점 복잡해진다

원인:

- loop 검토가 약함
- roles 검토가 약함

대응:

loop.md와 roles.md 기준으로 현재 결과물을 다시 검토해줘.
불필요한 복잡성, MVP 범위 초과, 구조 변경 위험을 찾아서 수정 제안을 해줘.

14. 최종 통합 템플릿

아래를 그대로 프로젝트에 넣고 시작하면 됩니다.

```
my-claude-project/
  CLAUDE.md
  docs/
    PRD.md
    ARCHITECTURE.md
    ADR.md
    UI_GUIDE.md
    TEST_PLAN.md
  HARNESS/
    router.md
    context.md
    loop.md
    roles.md
  TODO.md
```

```
LOG.md
source/
output/
```

15. 이 방법론의 핵심 결론

비개발자에게 가장 좋은 Claude Code 사용법은 이것입니다.

> **제품 방향은 docs로 고정한다.**
> **Claude 행동은 CLAUDE.md로 제한한다.**
> **실행 방식은 HARNESS로 통제한다.**
> **작업 순서는 TODO.md로 관리한다.**
> **작업 기억은 LOG.md로 유지한다.**

즉, 비개발자는 개발자가 되려는 것이 아닙니다.

비개발자가 해야 할 일은:

1. 무엇을 만들지 정의하고
2. 무엇을 만들지 않을지 제한하고
3. Claude가 지켜야 할 규칙을 만들고
4. 작업을 작게 나누고
5. 매번 기록하게 만드는 것

입니다.

Claude Code를 잘 쓰는 비개발자는 코드를 잘 쓰는 사람이 아니라, Claude가 일할 시스템을 잘 설계하는 사람이다.